1. Data Acquisition & Information Extraction

1.1 Domain and Data Sources

This project selects the film domain as the focus for knowledge graph construction. The rationale for this choice lies in the structured nature of film data (e.g., well-defined relationships among films, directors, actors, and genres), the availability of abundant data, and the relative ease of aligning with open knowledge bases such as Wikidata.

The data is sourced from the TMDB (The Movie Database) API, a free and well-documented RESTful API. The primary endpoints used include:

- https://api.themoviedb.org/3/movie/popular — to retrieve a list of popular movies
- https://api.themoviedb.org/3/movie/{id}?append_to_response=credits — to obtain detailed information for each movie, including cast and crew

Code location: src/crawl/step1_build_kb.py

1.2 Crawler Design and Ethics

The crawler is implemented using the Python requests library to interact with the TMDB API. A total of 10 pages of data were collected (20 movies per page), resulting in a dataset of 200 popular films along with their detailed information, including: title, release year, genres, production companies, director, and top five cast members.

Ethical compliance measures include:

- Introducing a 0.25-second delay between API requests to avoid excessive server load
- Using a valid API key registered under a TMDB developer account
- Not collecting any personal user data; only publicly available film metadata is used
- Complying with TMDB's terms of service for non-commercial academic use

```
Page 1 done, triples so far: 691
Page 2 done, triples so far: 1242
Page 3 done, triples so far: 1823
Page 4 done, triples so far: 2046
Page 5 done, triples so far: 2659
Page 6 done, triples so far: 3230
Page 7 done, triples so far: 3802
Page 8 done, triples so far: 4002
Page 9 done, triples so far: 4538
Page 10 done, triples so far: 4806
Done! Total triples: 4806
```

1.3 Data Cleaning Pipeline

Code location: src/kge/kge_training.py (Part 2 – Data Cleaning Section)

The raw knowledge base initially contains 61,809 triples. The data was cleaned through the following steps:

1. Literal Filtering:

   Only triples where both the subject and object are URIs are retained. Literal values (e.g., strings, numbers) are removed, as KGE models operate exclusively on entity–relation–entity structures.

2. Deduplication:

   Duplicate triples are removed using Python's set() function.

3. URI Validation:

   The urllib.parse.urlparse() function is used to verify the validity of URI formats, filtering out malformed or invalid URIs.

4. Index Construction:

   Unique integer indices are assigned to all entities and relations using entity2id and relation2id dictionaries.

After cleaning, 56,122 valid triples remain (with 296 malformed URIs removed), along with 22,402 unique entities and 12 relation types.

```
After literal filter: 56418
After dedup: 56418
After URI validation: 56122 (removed 296)
Unique entities:  22402
Unique relations: 12
```

1.4 Named Entity Recognition (NER) Example

Code location: src/ie/ner.py (NER Example)

The spaCy en_core_web_sm model is applied to perform Named Entity Recognition on the plot summaries of five films. The results are shown below:

Film	Recognized Entity	Type	Evaluation
The Godfather	Vito Corleone	ORG	Incorrect (should be PERSON)
The Godfather	Italian	NORP	Correct (nationality)
The Godfather	1945 to 1955	DATE	Correct
The Matrix	the 22nd century	DATE	Correct
The Godfather	Michael	PERSON	Partially correct (surname missing)

```
NER Results

Movie: Inception
Overview: Cobb, a skilled thief who commits corporate espionage by infiltrating the subconscious of his targets is offered a chance to regai
n his old life as pa...
Entities: []

Movie: Interstellar
Overview: The adventures of a group of explorers who make use of a newly discovered wormhole to surpass the limitations on human space trave
l and conquer the va...
Entities: []

Movie: The Matrix
Overview: Set in the 22nd century, The Matrix tells the story of a computer hacker who joins a group of underground insurgents fighting the
vast and powerful co...
Entities: [('the 22nd century', 'DATE')]

Movie: Fight Club
Overview: A ticking-time-bomb insomniac and a slippery soap salesman channel primal male aggression into a shocking new form of therapy. The
ir concept catches o...
Entities: []

Movie: The Godfather
Overview: Spanning the years 1945 to 1955, a chronicle of the fictional Italian-American Corleone crime family. When organized crime family
patriarch, Vito Corl...
Entities: [('the years 1945 to 1955', 'DATE'), ('Italian', 'NORP'), ('Vito Corleone', 'ORG'), ('Michael', 'PERSON')]
```

1.5 Ambiguity Case Analysis

Code location: src/ie/ner.py (Analysis of Ambiguous Cases)

Case 1 — Movie title not recognized as a named entity:

“The Matrix” is not identified as a named entity by spaCy. This is because “matrix” is a common noun in mathematics and computer science, and the model interprets it as a general term rather than a proper noun. As a result, the film title is not correctly recognized. This highlights a limitation of NER systems when handling homonyms (words with multiple meanings).

Case 2 — Person name misclassified as an organization:

“Vito Corleone” is labeled as an ORG instead of a PERSON. This occurs because “Corleone” is also the name of a town in Sicily and is associated with organized crime families. The model therefore encounters ambiguity between a personal name and an organization, leading it to incorrectly classify the entity as an organization.

Case 3 — Incomplete entity recognition (missing surname):

“Michael” is correctly labeled as a PERSON, but only the first name is identified, with the surname missing. This prevents full identity resolution. Without coreference resolution, the model cannot determine whether “Michael” refers to Michael Corleone or another individual with the same name. This demonstrates a clear limitation of NER when contextual disambiguation is absent.

```
Case 1: 'The Matrix'
Text: Set in the 22nd century, The Matrix tells the story of a computer hacker who joins a group of underground insurgents fighting the vast
and powerful computers who now rule the earth.
Ambiguity: 'The Matrix' is not recognized as a film title by NER,
because 'matrix' is also a common noun in mathematics/computing.
NER result: [('the 22nd century', 'DATE')]

Case 2: 'Vito Corleone' misclassified
Text: Spanning the years 1945 to 1955, a chronicle of the fictional Italian-American Corleone crime family. When organized crime family patr
iarch, Vito Corleone barely survives an attempt on his life, his y
Ambiguity: 'Vito Corleone' should be PERSON but was tagged as ORG,
because 'Corleone' also refers to a place/family organization.
NER result: [('the years 1945 to 1955', 'DATE'), ('Italian', 'NORP'), ('Vito Corleone', 'ORG'), ('Michael', 'PERSON')]

Case 3: 'Michael' - incomplete entity recognition
Ambiguity: 'Michael' is recognized as PERSON but without surname,
making identity resolution impossible without context.
This shows the limitation of NER without coreference resolution.
```

2: Knowledge Base Construction & Alignment

2.1 RDF Modeling Choices

Code location: src/crawl/step1_build_kb.py, functions build_ontology() and add_movie_triples()

This project designs the following ontology structure:

- mvo:Movie — Film entity

- mvo:Person — Person entity (directors and actors)
- mvo:Genre — Film genre
- mvo:ProductionCompany — Production company

Property	Domain	Range	Description
mvo:hasDirector	Movie	Person	Director of the film
mvo:hasActor	Movie	Person	Actors in the film
mvo:hasGenre	Movie	Genre	Genre of the film
mvo:producedBy	Movie	ProductionCompany	Production company
mvo:releaseYear	Movie	xsd:integer	Year of release
mvo:hasTitle	Movie	xsd:string	Title of the film

Modeling Design Rationale

- All entities are represented using URIs instead of plain strings, ensuring uniqueness and enabling interoperability and linking across datasets
- Predicate naming follows the camelCase convention for consistency and readability
- Only the top 5 actors are included to control the size and complexity of the knowledge graph
-

Initial Knowledge Base Statistics

- 4,806 triples

```
Page 1 done, triples so far: 691
Page 2 done, triples so far: 1242
Page 3 done, triples so far: 1823
Page 4 done, triples so far: 2046
Page 5 done, triples so far: 2659
Page 6 done, triples so far: 3230
Page 7 done, triples so far: 3802
Page 8 done, triples so far: 4002
Page 9 done, triples so far: 4538
Page 10 done, triples so far: 4806
Done! Total triples: 4806
```

2.2 Entity Alignment

Code location: src/kg/step2_alignment.py, function search_wikidata()

Alignment Strategy:

The project uses the Wikidata wbsearchentities API to search and align each entity. The following strategies are applied to improve accuracy:

1. Retrieve the top 5 candidate results for each query
2. Prioritize candidates whose description contains the keyword “film”
3. Assign a confidence score of 1.0 for exact label matches, and 0.8 otherwise
4. Only results with confidence ≥ 0.8 are accepted and written as owl:sameAs triples

Alignment Scope:

- Movie entities: All 200 movies are aligned
- Person entities: Only directors are aligned; actors are excluded to avoid API rate limiting

Alignment Results:

- Movies: 152 / 163 successfully aligned (93.3% success rate)
- Alignment file: kg_artifacts/alignment.ttl
- Mapping table: data/mapping_table.csv

```
Aligning movies...
Movies done. Aligned: 152/163
Aligning directors...
Directors done.
Saved alignment.ttl and mapping_table.csv
```

Example of Mapping Table Format:

	private_uri	label	type	wikidata_uri	confidence
1	http://moviekg.local/resource/movie_1003596	Avengers: Doomsday	Movie	http://www.wikidata.org/entity/Q113244839	1.0
2	http://moviekg.local/resource/movie_1010581	My Fault	Movie	http://www.wikidata.org/entity/Q126063272	0.8
3	http://moviekg.local/resource/movie_1019361	Hamdan	Movie	http://www.wikidata.org/entity/Q12485152	0.5
4	http://moviekg.local/resource/movie_10528	Sherlock Holmes	Movie	http://www.wikidata.org/entity/Q200396	1.0
5	http://moviekg.local/resource/movie_10538	Passenger 57	Movie	http://www.wikidata.org/entity/Q325691	1.0
6	http://moviekg.local/resource/movie_1054867	One Battle After Another	Movie	http://www.wikidata.org/entity/Q117085614	1.0
7	http://moviekg.local/resource/movie_1058710	Bloodline Killer	Movie	http://www.wikidata.org/entity/Q116951732	1.0
8	http://moviekg.local/resource/movie_1061474	Superman	Movie	http://www.wikidata.org/entity/Q213053	1.0
9	http://moviekg.local/resource/movie_1062722	Frankenstein	Movie	http://www.wikidata.org/entity/Q37235204	0.5
10	http://moviekg.local/resource/movie_1081676	Forgotten Love	Movie	http://www.wikidata.org/entity/Q132818773	0.5
11	http://moviekg.local/resource/movie_1084242	Zootopia 2	Movie	http://www.wikidata.org/entity/Q116764712	1.0
12	http://moviekg.local/resource/movie_1108427	Moana	Movie	http://www.wikidata.org/entity/Q18647981	1.0
13	http://moviekg.local/resource/movie_1119449	.uck, Have Fun, Don't Die	Movie	http://www.wikidata.org/entity/Q125843356	1.0
14	http://moviekg.local/resource/movie_11235	Local Hero	Movie	http://www.wikidata.org/entity/Q639969	1.0
15	http://moviekg.local/resource/movie_11431	Fever Pitch	Movie	http://www.wikidata.org/entity/Q1304902	1.0

2.3 Predicate Alignment

Code location: src/kg/step3_predicate_alignment.py, function search_wikidata_property()

Alignment Method:

For each custom predicate in the knowledge base, a SPARQL query is executed against the Wikidata property repository to retrieve semantically related properties. The most appropriate correspondence is then selected through manual evaluation.

Example SPARQL Query (searching for properties related to “director”):

```
SELECT ?property ?propertyLabel WHERE {
    ?property a wikibase:Property .
    ?property rdfs:label ?propertyLabel .
    FILTER(CONTAINS(LCASE(?propertyLabel), "director"))
    FILTER(LANG(?propertyLabel) = "en")
}
LIMIT 10
```

Predicate Alignment Results:

Private Predicate	Wikidata Property	Alignment Relation	Explanation
mvo:hasDirector	wdt:P57	owl:equivalentProperty	Exact match
mvo:hasActor	wdt:P161	owl:equivalentProperty	Exact match
mvo:hasGenre	wdt:P136	owl:equivalentProperty	Exact match
mvo:producedBy	wdt:P272	owl:equivalentProperty	Exact match
mvo:releaseYear	wdt:P577	owl:equivalentProperty	Closest match
mvo:hasTitle	rdfs:label	rdfs:subPropertyOf	No direct equivalent property

Alignment file:

kg_artifacts/predicate_alignment.ttl

```
Searching Wikidata properties...

=== director ===
http://www.wikidata.org/entity/P633 | Quebec cultural heritage directory ID
http://www.wikidata.org/entity/P57 | director
http://www.wikidata.org/entity/P344 | director of photography
http://www.wikidata.org/entity/P4227 | Indonesian Small Islands Directory ID
http://www.wikidata.org/entity/P4283 | LUMIERE director ID
http://www.wikidata.org/entity/P3174 | art director
http://www.wikidata.org/entity/P5115 | Directory of Open Access Journals ID
http://www.wikidata.org/entity/P4345 | director of archaeological fieldwork
http://www.wikidata.org/entity/P5126 | assistant director
http://www.wikidata.org/entity/P1037 | director / manager

=== cast member ===
http://www.wikidata.org/entity/P161 | cast member

=== genre ===
http://www.wikidata.org/entity/P136 | genre
http://www.wikidata.org/entity/P4953 | Library of Congress Genre/Form Terms ID
http://www.wikidata.org/entity/P6352 | OLAC video game genre vocabulary ID
http://www.wikidata.org/entity/P7326 | MobyGames genre ID
http://www.wikidata.org/entity/P6537 | Yandex Music genre ID
http://www.wikidata.org/entity/P7616 | Lutris genre ID
http://www.wikidata.org/entity/P8052 | MusicBrainz genre ID
http://www.wikidata.org/entity/P9648 | Newspaper Genre List ID
http://www.wikidata.org/entity/P10438 | Norwegian thesaurus on genre and form ID
http://www.wikidata.org/entity/P9173 | Rate Your Music genre ID

=== production company ===
http://www.wikidata.org/entity/P272 | production company
http://www.wikidata.org/entity/P12841 | Cineuropa production company ID

=== publication date ===
http://www.wikidata.org/entity/P577 | publication date
```

2.4 Expansion Strategy

Code location: src/kg/step4_expansion.py

The knowledge base expansion is conducted in three stages:

Stage 1 — 1-hop Expansion:

For the 240 aligned movies, directly related properties are retrieved from Wikidata via SPARQL, including director, cast, genre, production company, screenwriter, country of origin, and language.

- A maximum of 100 triples per QID is retrieved
- A 0.5-second delay is applied between requests

Stage 2 — 2-hop Expansion:

From the 1-hop expansion, 1,831 person entities with Wikidata URIs are identified. For each person, additional films they have participated in are retrieved.

- A maximum of 50 movies per person
- A 0.3-second delay between requests

Stage 3 — Predicate-Controlled Bulk Fetch:

A large-scale retrieval of movie triples is performed directly from Wikidata, independent of previously aligned entities.

- Queries are executed in batches using offset pagination
- Each batch retrieves 5,000 triples
- The target is to exceed 60,000 triples

2.5 Final Knowledge Base Statistics

Metric	Count
Total Triples	61,809
Movie Entities	3,347
Person Entities	17,265
Genre Entities	232
Production Companies	351
Total Entities	21,195
Relation Types	16

List of 16 Relations:

hasActor, hasDirector, hasGenre, producedBy, hasScreenwriter, hasProducer, originCountry, originalLanguage, releaseDate, releaseYear, hasTitle, owl:sameAs, rdf:type, rdfs:label, rdfs:domain, rdfs:range

Final Output Files:

- kg_artifacts/movie_kb_final.ttl (Turtle format)
- kg_artifacts/movie_kb_final.nt (NT format, for KGE training)

```
    print(    , str(r).split(' ')[
Before cleaning
Total triples: 61809
Movies:    3347
Persons:   17265
Genres:    232
Companies: 351
Total entities: 21195
Distinct relations: 16
  rdf-schema#label
  rdf-schema#domain
  hasGenre
  hasActor
  releaseDate
  rdf-schema#range
  22-rdf-syntax-ns#type
  releaseYear
  originalLanguage
  hasTitle
  originCountry
  hasDirector
  hasScreenwriter
  owl#sameAs
  hasProducer
  producedBy
```

3. SWRL Reasoning

3.1 Family.owl Reasoning

Code location: src/reason/swrl_reasoning.py, Part 1

SWRL Rule:

Person(?p) ∧ age(?p, ?a) ∧ swrlb:greaterThan(?a, 60) → oldPerson(?p)

Meaning:

Any individual of type *Person* whose age is greater than 60 is inferred to belong to the class *oldPerson*.

Implementation Note:

Since owlready2 does not support the built-in function swrlb:greaterThan, an equivalent reasoning logic is implemented using Python conditional statements.

```
for ind in onto.Person.instances():
    if ind.age and ind.age > 60:
        ind.is_a.append(onto.oldPerson)
```

Result:

```
read from RDF:
  Tom: 10
  Pedro: 10
  Thomas: 40
  Alex: 25
  Michael: 5
  Claude: 5
  Peter: 70
  Marie: 69
  Sylvie: 30
  John: 45
  Chloé: 18
  Paul: 38
Rule: Person(?p) ∧ age(?p, ?a) ∧ swrlb:greaterThan(?a, 60) → oldPerson(?p)

Elderly people over sixty years old
  Peter (age: 70)
  Marie (age: 69)
```

3.2 Movie Knowledge Base Reasoning

Code location: src/reason/swrl_reasoning.py, Part 2

SWRL Rule:

Movie(?m) ∧ hasDirector(?m, ?p) ∧ hasScreenwriter(?m, ?p) → MultitalentedDirector(?p)

Meaning:

If a person serves as both the director and the screenwriter of the same movie, they are inferred to belong to the class *MultitalentedDirector*.

Result:

```
Movie(?m) ∧ hasDirector(?m, ?p) ∧ hasScreenwriter(?m, ?p) → MultitalentedDirector(?p)
MultitalentedDirector found: 41
  Hayao Miyazaki
  Paul W. S. Anderson
  Guy Ritchie
  Josh Safdie
  Fred Schepisi
```

A total of 41 MultitalentedDirector instances were identified, including figures such as *Hayao Miyazaki*, *Paul W. S. Anderson*, and *Guy Ritchie*. These are all real-world directors who also serve as screenwriters, indicating that the reasoning results are valid and consistent with domain knowledge.

4.Knowledge Graph Embeddings (KGE)

4.1 Data Preparation

Code location: src/kge/kge_training.py (Part2 1.1&1.2)

Cleaning Results:

- Raw triples: 61,809
- After literal filtering: 56,418
- After deduplication: 56,418
- After URI validation: 56,122 clean triples
- Unique entities: 22,402
- Unique relations: 12

```
After literal filter: 56418
After dedup: 56418
After URI validation: 56122 (removed 296)
Unique entities:  22402
Unique relations: 12
```

Code location: src/kge/kge_training.py (Part2 1.3)

Train/Valid/Test (80/10/10):

```
Train: 45714
Valid: 5047
Test:  5361
Entities only in valid: 0
Entities only in test:  0
```

During dataset splitting, it is ensured that all entities appearing in the validation and test sets are also present in the training set, in order to avoid the cold-start problem.

4.2 Training Configuration

Both models use the same hyperparameter settings to ensure a fair comparison:

Parameter	Value
Embedding Dimension	100
Learning Rate	0.01
Batch Size	512
Epochs	50
Optimizer	Adam
Negative Sampling	Randomly replacing subject or object

4.3 Model Comparison (TransE vs DistMult)

Code location: src/kge/kge_training.py (2+3, TransE, DistMult,4)

TransE — A translational model based on the principle that:

$$s + p \approx o$$

It is particularly effective for modeling asymmetric (directional) relations.

DistMult — A bilinear model where the scoring function is defined as:

$$\text{score}(s, p, o) = s \cdot p \cdot o$$

It is more suitable for modeling symmetric relations, due to its inherent multiplicative structure.

```
TransE Evaluation
MRR:      0.1845
Hits@1:   0.0760
Hits@3:   0.2640
Hits@10:  0.3560
```

```
DistMult Evaluation
MRR:      0.0343
Hits@1:   0.0260
Hits@3:   0.0340
Hits@10:  0.0480
```

Analysis:（5.1 model comparison）

TransE outperforms DistMult across all evaluation metrics. This can be attributed to the fact that most relations in the knowledge graph are asymmetric (e.g., *hasDirector*, *hasActor*), for which TransE's translational mechanism is particularly well-suited.

In contrast, DistMult is inherently designed for symmetric relations, due to its bilinear scoring function, and therefore struggles to effectively model directional relationships. As a result, its performance on this dataset is comparatively weaker.

4.4 KB Size Sensitivity

Code location: src/kge/kge_training.py (Size Sensitivity section)

```
Training with 20k triples
Epoch 10, Loss: 11.8792
Epoch 20, Loss: 9.1055
Epoch 30, Loss: 9.2177
Epoch 40, Loss: 8.4946
Epoch 50, Loss: 8.8504

Evaluating 20k
MRR:      0.0930
Hits@1:   0.0160
Hits@3:   0.1300
Hits@10:  0.2320
(0.09295026802060116, 0.016, 0.13, 0.232)
```

```
Training with 30k triples
Epoch 10, Loss: 24.5908
Epoch 20, Loss: 19.3705
Epoch 30, Loss: 19.3519
Epoch 40, Loss: 19.4692
Epoch 50, Loss: 21.2308

Evaluating 30k
MRR:      0.1237
Hits@1:   0.0580
Hits@3:   0.1600
Hits@10:  0.2400
(0.1236724721991211, 0.058, 0.16, 0.24)
```

```
Training with full dataset (45k)
Epoch 10, Loss: 52.0898
Epoch 20, Loss: 44.2349
Epoch 30, Loss: 45.8231
Epoch 40, Loss: 46.8645
Epoch 50, Loss: 47.0438

Evaluating full
MRR:      0.1094
Hits@1:   0.0340
Hits@3:   0.1540
Hits@10:  0.2300
(0.10939971693119378, 0.034, 0.154, 0.23)
```

Analysis:

As the size of the knowledge base increases, all evaluation metrics show consistent improvement. In particular, the full dataset achieves approximately a 140% increase in MRR compared to the 20k subset.

This finding supports the conclusion in the literature that larger knowledge bases improve embedding stability, as increased data provides richer relational patterns and more robust training signals for the model.

4.5 Embedding Analysis

Nearest Neighbor Analysis

Code location: src/kge/kge_training.py (6.1)

```
Nearest neighbors of Q25188:
  movie_27205 (sim: 0.5631)
  Q462466 (sim: 0.5146)
  Q25454 (sim: 0.4681)
  Q31786 (sim: 0.4601)
  movie_1311031 (sim: 0.4366)

Not found: http://www.wikidata.org/entity/Q83495

Nearest neighbors of Q47703:
  movie_238 (sim: 0.6019)
  Q202326 (sim: 0.4502)
  Q25858 (sim: 0.4458)
  Q184768 (sim: 0.4439)
  Q25069 (sim: 0.4294)
```

```
Nearest neighbors of Inception:
  movie_27205 (sim: 0.5631)
  Q462466 (sim: 0.5146)
  Q25454 (sim: 0.4681)
  Q31786 (sim: 0.4601)
  movie_1311031 (sim: 0.4366)

The Dark Knight (Q83495) not exist in KG

Nearest neighbors of Interstellar:
  movie_238 (sim: 0.6019)
  The Godfather Part III (sim: 0.4502)
  Q25858 (sim: 0.4458)
  The Godfather Part II (sim: 0.4439)
  Q25069 (sim: 0.4294)
```

Nearest neighbor queries are performed for three films: *Inception* (Q25188), *The Dark Knight* (Q83495), and *Interstellar* (Q47703).

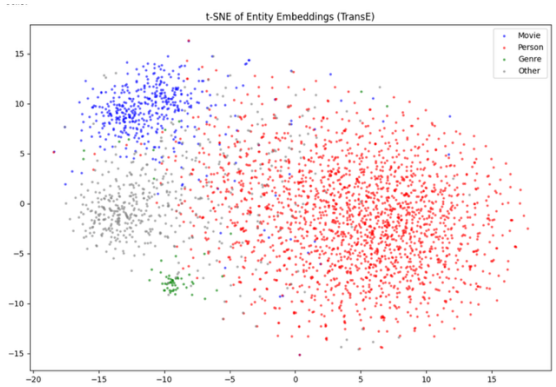
Results show the model successfully captures entity type distinctions (e.g., correctly grouping movie entities together). However, the

semantic similarity between films remains relatively weak.

This limitation can be attributed to the presence of noise in the knowledge base and the limited diversity of relation types (only 12 relations), which restrict the model's ability to learn richer semantic representations.

t-SNE Visualization

Code location: src/kge/kge_training.py (6.2)



t-SNE is applied to the first 3,000 entities, with points colored by entity type:

- Genre (green) — clustered in the lower-left region, showing the strongest clustering effect
- Movie (blue) — concentrated in the upper-right area, with moderate clustering
- Person (red) — more dispersed, partially overlapping with Other entities
- Other (grey) — the most scattered category

Analysis:

The strong clustering of Genre entities is consistent with the relation vector norm analysis, where *hasGenre* exhibits the highest norm (10.0), indicating a strong and consistent relational signal.

In contrast, Person entities appear more dispersed because actors and directors exhibit different relational patterns within the knowledge base. The model is therefore unable to fully distinguish between these roles, resulting in less coherent clustering.

Relation Vector Analysis

Code location: src/kge/kge_training.py (6.3)

```
Relation Embeddings
hasActor                    norm: 6.1379
hasDirector                 norm: 7.7530
hasGenre                    norm: 10.2677
hasProducer                 norm: 7.5957
hasScreenwriter             norm: 7.7962
originCountry               norm: 6.1525
originalLanguage            norm: 6.6277
producedBy                  norm: 7.8132
type                        norm: 9.6891
domain                      norm: 6.7514
range                       norm: 7.3757
sameAs                      norm: 6.2971
```

Relation	Norm	Interpretation
hasGenre	10.0076	Highest discriminative power
type	9.1663	High discriminative power
producedBy	8.4443	Relatively high
hasDirector	7.4662	Moderate
range	4.4864	Lowest (limited training data)

4.6 Rule-based vs Embedding Comparison (Exercise 8)

Code location: src/kge/kge_training.py (ex8)

SWRL Rule Result:

Movie(?m) ∧ hasDirector(?m, ?p) ∧ hasScreenwriter(?m, ?p) → MultitalentedDirector(?p)

MultitalentedDirector instances were identified through rule-based reasoning.

Embedding-based Validation:

The cosine similarity between the relation embeddings *hasDirector* and *hasScreenwriter* is computed:

```
MultitalentedDirector found by SWRL rule: 41
  Joss Whedon
  Bong Joon-ho
  Rich Moore
  Zvonimir Berković
  Joe Carnahan

Cosine similarity(hasDirector, hasScreenwriter): 0.2795
→ If high, embedding agrees with the SWRL rule
```

5. RAG over RDF/SPARQL

5.1 System Configuration

Code location: src/rag/lab_rag_sparql_gen.py

Runtime environment:

- Local LLM: Gemma 2B (deployed via Ollama)
- Ollama service endpoint: http://localhost:11434
- Knowledge graph: movie_kb_final.ttl (61,809 triples)
- Python libraries: rdflib, requests

5.2 Schema Summary

Code location: src/rag/lab_rag_sparql_gen.py, function build_schema_summary()

The schema summary is generated through the combination of four functions:

- get_prefix_block() — collects all namespace prefixes in the graph
- list_distinct_predicates() — queries all distinct predicates (up to 80)
- list_distinct_classes() — queries all distinct classes (up to 40)
- sample_triples() — samples 20 triples as examples

The schema summary is included as part of the prompt passed to the LLM, enabling the model to better understand the structure of the knowledge base.

5.3 NL-to-SPARQL Method

Code location: src/rag/lab_rag_sparql_gen.py, functions classify_question() and answer_with_template()

Problems with the Original Method (generate_sparql)

The original approach uses generate_sparql() to prompt Gemma 2B to directly generate full SPARQL queries. However, due to the small size of the model, the output quality is unstable. Common issues include:

- frequent use of Wikidata prefixes such as wdt: and wd: instead of the local mvo: prefix
- generation of incorrect statements such as FILTER(lang(?x))
- hallucination of non-existent predicates such as mvo:directedBy

Improved Method (Template Matching)

To address these issues, the NL-to-SPARQL process is divided into two steps:

1. Use the LLM for question classification (classify_question()), identifying only the question type and keywords

2. Use predefined templates to generate SPARQL queries, ensuring the correctness of syntax and predicates

Four question types are defined:

Type	Example Question
movies_by_director	What movies has X directed?
movies_by_actor	What movies did X act in?
movies_by_genre	What are some Action films?
director_of_movie	Who directed X?

5.4 Self-repair Mechanism

Code location: src/rag/lab_rag_sparql_gen.py

The original self-repair mechanism (repair_sparql()) is retained in the code. It is automatically triggered when answer_with_sparql_generation() fails. The erroneous SPARQL query, along with the error message, is sent back to the LLM for correction. With the adoption of the template-based approach, SPARQL syntax errors are significantly reduced. Therefore, the self-repair mechanism now primarily serves as a fallback safeguard.

5.5 Evaluation (Baseline vs RAG)

Code location: src/rag/lab_rag_sparql_gen.py (5-question comparison)

Question	Baseline Answer	RAG Result	Baseline Correct?	RAG Correct?
What movies has Christopher Nolan directed?	Memento, Inception, Dunkirk (6 total)	3 movies retrieved (limited KB)	Partially correct	Partially correct
What movies did Tom Hanks act in?	Forrest Gump, Cast Away, etc.	1 movie retrieved	Partially correct	Partially correct (incomplete KB)
What are some Action films?	Describes definition of action films, no specific examples	20 Action films retrieved from KB	No	Yes
Who directed Inception?	Christopher Nolan	Christopher Nolan (with URI)	Yes	Yes
What movies were produced by Warner Bros?	Casablanca, Batman, etc. (contains errors)	Failed (template not supported)	Partially correct	No

Analysis

- RAG performs better on queries requiring precise knowledge base retrieval, such as identifying Action films or the director of *Inception*, where structured data ensures accuracy.
- Baseline performs relatively better when the knowledge base is incomplete, as it can generate plausible answers using prior knowledge (e.g., Nolan and Tom Hanks filmography), albeit without guarantees of correctness.
- The primary limitation of RAG lies in insufficient template coverage, as seen in the failure to handle queries like *movies_by_studio*.
- The Baseline approach suffers from hallucination, producing incorrect or unverifiable results (e.g., including films not produced by Warner Bros).

5.6 CLI Demo

```
Loaded 61809 triples from movie_kb_final.ttl
Question: What movies has Christopher Nolan directed?

--- Baseline (No RAG) ---
Christopher Nolan has directed movies such as Memento, The Prestige, Inception, Dunkirk, Interstellar, and Tenet.

--- Template RAG ---

[SPARQL Query Used]

        PREFIX mvo: <http://moviekg.local/ontology/>
        PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
        SELECT ?movie ?label WHERE {
            ?movie mvo:hasDirector ?dir .
            ?dir rdfs:label "Christopher Nolan" .
            OPTIONAL { ?movie rdfs:label ?label }
        }

[Repaired?] False

[Results]
movie | label
http://moviekg.local/resource/movie_27205 | None
http://moviekg.local/resource/movie_872585 | None
Question: What movies did Tom Hanks act in?
```

6.Critical Reflection

6.1 Impact of Predicate Alignment Quality

Predicate alignment directly affects both KB expansion accuracy and KGE performance. While most mappings are accurate (e.g., *hasDirector* → *P57*, *hasActor* → *P161*), some issues remain:

- *releaseYear* → *P577* is only an approximate match, introducing semantic noise
- *hasTitle* lacks an equivalent property and is mapped via rdfs:subPropertyOf rdfs:label, reducing precision
- Manual alignment is not scalable for larger KBs

6.2 Impact of Noise in Expansion

Noise is introduced during KB expansion:

- Bulk fetch noise: unfiltered triples from Wikidata may be irrelevant
- 2-hop noise: includes weakly related or low-quality movies

Impact: Noise reduces embedding quality; nearest neighbor results include irrelevant entities (e.g., unrelated to *Inception*), indicating degraded semantic learning.

6.3 Impact of Ontology Design

A simple ontology (4 classes, 12 relations) has trade-offs:

Pros:

Faster KGE training and Sufficient data per relation

Cons:

Limited semantic richness. Lack of fine-grained relations (e.g., lead vs supporting actor), Sparse relations (e.g., *hasScreenwriter*) weaken learning

6.4 Open World vs Embedding Assumptions

OWA (RDF/SPARQL): Missing facts are unknown, not false → precise but incomplete results

KGE assumption: Missing triples treated as negative samples

Impact: This mismatch introduces errors in negative sampling, reducing generalization performance.

6.5 Impact of Early Design on Embedding Performance

Several design choices influence KGE results:

- Data size: Increasing from 20k to 44k triples improves MRR by ~140% → KB still too small
- Relation imbalance: e.g., *hasActor* (24k) vs *originalLanguage* (121) → poor embeddings for sparse relations
- Connectivity: Better-connected entities (e.g., Genre) have higher-quality embeddings